TERMINAL-BENCH 3 · TASK CONTRIBUTION

Certificate verification under a service-level objective

Design rationale, measurements, and iteration log

Renan Brito Cano Butkeraites · category `ML / Evaluation` · slug `certificate-verifier-slo`

A verifier that is right 99% of the time is worthless as a trust layer. This task is built around that fact: a service must return the exactly correct verdict on every submission in a graded stream *and* stay inside a latency budget, and the two requirements pull in opposite directions.

1. The task

Submissions arrive as claimed optimality certificates for mixed-integer linear programs in the fixed form

```
minimise    c'x
subject to  Ax ⩾ b,  0 ⩽ x ⩽ u,  x_j integer for j in I
```

The dual of the continuous relaxation is `max b'y - u'w` subject to `A'y - w ⩽ c`, `y, w ⩾ 0`. Weak duality gives `b'y - u'w ⩽ c'x` for every primal feasible `x`, and that bound is valid for the integer problem too, since every integer feasible point is relaxation feasible. A certificate is therefore the triple `(x, y, w)` together with the claims made about it.

The agent is given a working service that is correct on well-conditioned input, plus a public submission set with answers. A judge sidecar replays a graded stream against the service and records what it answered. The correct verdicts are never on the machine.

2. Why the verdict is only well defined in exact arithmetic

Two reasonable floating-point implementations of the same inner product disagree:

```
c = [1e16, 1, -1e16, 1]        x = [1, 1, 1, 1]

naive left-to-right float sum  →  2.0
numpy.dot (pairwise summation) →  1.0
exact rational                 →  2
```

So “the objective value” is not a property of the submission until the arithmetic is pinned down. The specification pins it to exact rational arithmetic, and every quantity on the wire is a decimal *string* rather than a JSON number — `float("0.1")` is `3602879701896397/36028797018963968`, so a parser that goes through binary floating point has lost the value before any arithmetic happens.

There is no tolerance parameter anywhere in the specification. A constraint violated by one part in a billion is violated. Any tolerance would be a hole an adversarial submitter walks through.

Rejection precedence

When several conditions fail, the reported reason is the first in a declared order, so the verdict is a deterministic function of the submission rather than an artefact of evaluation order:

```
dimension_mismatch → primal_infeasible → integrality_violated
→ objective_mismatch → dual_infeasible → bound_mismatch
→ optimality_claim_unsupported
```

3. The crux, measured

Full verification path, no early exit, one core of the development machine. The environment has open internet, so the agent will reach for `gmpy2`; the budget is sized against GMP-backed rationals, not against the standard library.

Instance	`float64 (numpy)`	`gmpy2 mpq`	`stdlib Fraction`	ratio
50 × 40	0.013 ms	–	3.66 ms	290×
200 × 150	0.031 ms	8.02 ms	46.8 ms	1490×
400 × 300	0.071 ms	35.5 ms	190 ms	2683×

Exact arithmetic is two to three orders of magnitude slower than float, and `gmpy2` recovers only about 5.5× of that. Neither pure strategy survives:

Per-submission cost, median over the graded stream, per-submission minimum of three repeats (`verifier-core/scripts/benchmark_paths.py`):

Strategy	Median	Per second, 1 core
float everywhere	18.9 ms	53
stdlib `Fraction` everywhere	325 ms	3
`gmpy2` everywhere	193 ms	5
float screen + certified bound, lazy exact fallback	**22.0 ms**	**46**

What the run actually does

Those per-submission costs understate the separation, because the graded run offers a fixed rate rather than waiting politely. At 400 submissions and 200 requests per second against four cores, an exact-everywhere service is overloaded about two and a half times, and an overloaded queue does not degrade linearly — it runs away. Every row below is a measured end-to-end run through the real judge and the real verifier:

Service	Verdicts	Achieved rate	p99	Reward
reference solution (hybrid, 4 workers)	**all correct**	199 /s	**62 ms**	**1**
exact everywhere (`gmpy2`, 4 workers)	all correct	48 /s	1838 ms	0
starting service (float, 1 worker)	**3.7% wrong**	128 /s	536 ms	0
cheat oracle (no verification at all)	**54% wrong**	200 /s	16 ms	0

The raw throughput ratio between the hybrid and exact-everywhere is 8.8×; the observed latency ratio is **30×**. The 250 ms budget sits with four times headroom below it and seven times above — wide enough that a slower or faster CI runner does not flip the result, which is what the `deterministic_reproducible` criterion is protecting against.

The last row is the one worth staring at. The cheat oracle is *four times faster than the reference solution* and still scores zero. Speed bought without verification buys nothing.

> **Where 80% becomes zero.** Building the service is not the hard part; a capable model does that comfortably. The hard part is the error bound on the float screen. Too loose and the service escalates to exact arithmetic on everything and misses the rate. Too tight and it accepts one wrong verdict — and one wrong verdict fails the run.

4. Sizing the design window

A hybrid escalates to exact arithmetic exactly on the submissions a float screen cannot decide, and the adversarial submissions are by construction inside any plausible float tolerance. So the adversarial share of the stream sets the fallback rate, and the hybrid's advantage over exact-everywhere is roughly its reciprocal.

The share was set to **8.3%**: large enough that a float-everywhere service reliably gets verdicts wrong, small enough that the intended solution keeps real headroom. This is the most consequential number in the stream generator and it is documented as such in the source.

> **A measurement lesson, recorded because it changed the answer.** Earlier estimates of this window ranged from 2.1× to 11.7× across ad-hoc timing scripts, and none survived scrutiny. Two causes. Timing a rejected submission measures a short-circuit, not the work; and the generator guarantees one submission of each kind before sampling by weight, so a short stream carries 12% accepted submissions against a nominal 45% — which makes a mean over a small sample badly underestimate the exact path. The harness now gates timings on cross-implementation verdict agreement, takes per-submission minima, and reports the breakdown by decision point instead of averaging it away.

5. Anti-cheat design

The verdicts never reach the agent

The judge sidecar is a dumb recorder: it replays submissions and logs what came back, and it does not know right from wrong. Ground truth lives only in the verifier image, which is built from `tests/` and runs in a separate container after the agent container has been torn down. The agent container does not mount the verdict-log volume; it reaches the judge over HTTP only.

A fresh stream per run

The agent sees every submission — they arrive at its service — so the stream cannot be secret. That opens a concrete attack: record a run, solve it offline with slow exact arithmetic, serve a cache on the next one. A merged Terminal-Bench task is public, so a stream derived only from a baked-in secret would be predictable from the repository itself. Each graded run therefore draws a nonce at run time and seals it inside the log's HMAC chain; the verifier regenerates from it, and nothing can predict it beforehand.

The attacks, run

`cheat/solve.sh` mounts four of them at once and must score zero:

Attack	Outcome
Write the verdict log	Writes a local file of the same name. The judge's `/shared` is a named volume it alone mounts.
Write the reward	Same: `/logs/verifier` belongs to the verifier image, built and run after the agent's container is gone.
Cache across runs	80 answers absorbed, **zero hits** on the graded stream. This is the nonce doing its work.
Guess the majority class	200 /s, p99 16 ms, and **217 of 400 verdicts wrong**.

A leak found and closed during construction

The base coefficients originally used only denominators whose prime factors are 2 and 5, so every value had a finite decimal expansion. Fraction syntax such as `"41/3"` then appeared on exactly one mutation, which made its presence a free tell for that verdict. Denominators 3, 7, 9 and 11 are now mixed throughout the stream.

6. Iteration log

Recorded because the process is part of what the task is worth.

A discarded first direction

The task was originally built on target localisation from interval distances, over a method the author published in 2018. Two measurements killed it. First, a full reimplementation of the paper's three optimisation models took roughly four hours and passed 105 tests — direct evidence that "implement or repair an optimisation method" does not stop a frontier model. Second, a 25,600-trial sweep showed that the apparent crux, a 50% non-convergence rate, was not numerical failure at all: a direct geometric test found the model to be *genuinely infeasible* on about 40% of noisy instances, with the real numerical failure rate at 2%. The drafted task asked for something impossible.

That work is not lost. The feasibility condition the 2018 models presuppose is never stated in the paper, which is a separate and publishable finding.

Corrections made during construction of the present task

- **Adversarial share retuned from 13% to 8.3%** once the hybrid-versus-exact window was computed rather than assumed.
- **Fraction-syntax leak closed**, as above.
- **Over-correction caught by the test suite.** Spreading fraction syntax through the stream made the starting service crash on 96% of submissions, because `float("41/3")` raises. That destroyed the premise that the service is correct on well-conditioned input. It now parses fraction syntax and remains float-based — wrong for the intended reasons rather than by crashing.
- **Restart durability dropped.** The plan called for the judge to `SIGKILL` a service worker mid-stream. The agent container shares the judge's network namespace, not its PID namespace, so the judge cannot signal the process, and any `/admin/restart` endpoint would be a no-op the agent controls. Replaced by resilience to abrupt connection resets, which the judge drives end to end. This is a weaker requirement and is recorded as such.

7. Validation status

Gate	Result
Static checks (all 22)	pass
Specification test suite	108 pass
`harbor run --agent oracle`	reward 1.0
`harbor run --agent nop`	reward 0.0
`cheat/solve.sh`	reward 0.0

8. Open risks

1. **The task may still be too easy.** Building a robust FastAPI service is strong model territory. The wager is that adversarial correctness *under a latency budget* is what breaks, not the service itself. Only a real trial answers this, and the trial is the gate the schedule is built around.
2. **Latency thresholds are a classic source of CI non-determinism**, which the `deterministic_reproducible` criterion rejects. Mitigated by grading work completed within a budget rather than raw wall clock, and by keeping a wide margin between the reference solution and the threshold.
3. **The window depends on the adversarial share holding across instance sizes.** The graded configuration is 200 × 150; changing it means re-measuring, because the share sets the escalation rate and the escalation rate sets the window.
4. **The judge must not become the bottleneck.** It generates every payload itself, and an early version managed two a second against a target of two hundred — it now pre-generates before the clock starts, and drives 200 /s against a null service, but that ceiling is a property of this hardware and should be re-checked on the CI backend.

Measurements reproduce via `verifier-core/scripts/benchmark_arithmetic.py`. Verdict semantics are specified in `SPEC.md` and implemented as an executable specification in `verifier-core/src/certverify/`.